Concept of Operations

countdown: An X Window Countdown Timer

Countdown Software Project

Prepared for: Users and Maintainers of `countdown`

Prepared by: J. R. Evans

Document identifier:	CD-CONOPS-001
Revision:	1.0
Date:	June 13, 2026
Status:	Released
Governing standard:	IEEE Std 1362-1998 (ConOps)

This document is prepared in accordance with the principles of rational documentation articulated in D. L. Parnas and P. C. Clements, "A Rational Design Process: How and Why to Fake It," IEEE Transactions on Software Engineering, vol. SE-12, no. 2, pp. 251–257, February 1986.

Contents

1 Scope

1.1 System identification

This Concept of Operations (ConOps) describes `countdown`, a small single-window graphical application for the X Window System. Given a fixed future instant — by default `2026-08-19 08:00:00` in local time — the program displays the time remaining until that instant and updates the display live until it is reached.

1.2 Document purpose

The ConOps states, in the language of the user, what `countdown` is for, how the situation looks without it, why it is worth building, and how a user interacts with it. It is the topmost document in the `countdown` documentation set; the Software Requirements Specification (CD-SRS-001), Software Design Document (CD-SDD-001), and Software Test Plan (CD-STP-001) refine and verify the intent recorded here.

1.3 A note on rational documentation

Following Parnas and Clements, this document is written as though the concept had been arrived at by an orderly process. The real path was the usual mixture of trial and revision; the value of presenting an idealised account is that a reader can understand the system without retracing every false start. The same fiction is maintained, deliberately, throughout the documentation set.

2 Referenced documents

Reference	Title
IEEE Std 1362-1998	Guide for Information Technology—System Definition—Concept of Operations Document
ISO/IEC/IEEE 29148:2018	Systems and software engineering—Life cycle processes—Requirements engineering
Parnas & Clements, 1986	A Rational Design Process: How and Why to Fake It
Knuth, 1984	Literate Programming, *The Computer Journal* 27(2)
CD-SRS-001	Software Requirements Specification: countdown
CD-SDD-001	Software Design Document: countdown
CD-STP-001	Software Test Plan: countdown
X11R7 / Xlib	*Xlib—C Language X Interface*, X Consortium
POSIX.1-2017	IEEE Std 1003.1-2017 (`time`, `mktime`, `select`)

3 Current system or situation

3.1 Background

People frequently wish to mark the approach of a specific moment: the start of an event, a deadline, a launch, a celebration. In the absence of a dedicated tool the countdown is performed by mental arithmetic against a wall clock or calendar, or by a web page of uncertain provenance that requires a network connection and a browser.

3.2 Description of the current situation

On a typical desktop running the X Window System the user today has:

- the `date` command, which prints the current time but performs no subtraction toward a target;
- general-purpose calculators, into which the user must transcribe and difference two timestamps by hand;
- online countdown widgets, which depend on a browser, a network path, and a third party's continued operation.

3.3 Problems and limitations

The manual approach is error-prone (time-zone and daylight-saving mistakes are common), is not continuously updated, and gives no glanceable visual artefact that can be left running on the desktop. The online approach surrenders privacy and availability to a remote service for a computation the local machine can trivially perform.

4 Justification for change

A countdown to a known instant is a self-contained computation requiring only the system clock and the local time-zone rules. A small native program can perform it correctly, continuously, and offline, presenting a single uncluttered window. The justification for building `countdown` is therefore:

- **Correctness:** time-zone and daylight-saving handling is delegated to the operating system's `mktime`, eliminating a common class of manual error.
- **Continuity:** the display advances automatically, without user action, at sub-second resolution.
- **Independence:** no network, browser, or third-party service is involved; the program runs wholly on the local host.
- **Transparency:** the program is a literate document, so its behaviour can be read and audited in full.

5 Proposed system

5.1 Overview

`countdown` is a single executable that opens one window on the X Window System. The window shows the target instant, the remaining time expressed as days and a `HH:MM:SS` clock, a status line, and a one-line hint listing the available keys. The user starts and pauses the countdown from the keyboard or mouse and quits when finished.

5.2 Operational environment

The program runs on any POSIX host providing an X server (Linux with an X display, or macOS with XQuartz). It links only against the core Xlib client library; no widget toolkit is required. It consumes negligible CPU because it sleeps on `select` between updates rather than polling.

5.3 High-level capabilities

1. Count down to a default instant of `2026-08-19 08:00:00`, or to any instant supplied on the command line.
2. Display the remaining time live, updating at least once per second.
3. Let the user start, pause, resume, and quit interactively.
4. Announce arrival when the target instant is reached, and never display a negative remaining time.

5.4 Modes of operation

The program has three user-visible modes: *armed* (started but not yet running — the initial state, showing the full duration); *running* (counting down live); and *paused* (the figure is frozen at the value captured when the user paused). A fourth, terminal condition, *arrived*, is entered when the remaining time reaches zero.

6 Operational scenarios

6.1 Scenario 1: Counting down to the default instant

Dana wants a visible countdown to the morning of 19 August 2026 left running on her desktop. She launches `countdown` with no arguments. A window appears showing *Target: Wed 19 Aug 2026 08:00:00* and the full remaining duration with the status *ready*. She presses the space bar; the status changes to *running* and the seconds begin to tick down. She leaves the window in a corner of her screen.

6.2 Scenario 2: A custom target

Raj is timing a 25-minute presentation rehearsal that should end at 14:30. He runs `countdown "2026-06-13 14:30:00"`. The window opens with that instant as the target. Part-way

through he is interrupted; he clicks the window to pause, deals with the interruption, then clicks again to resume.

6.3 Scenario 3: Reaching the target

As the target instant arrives, the remaining figure reaches `0 d 00:00:00` and the status line changes to *THE MOMENT HAS ARRIVED*. The figure does not go negative. The user observes the message and presses `q` to quit.

6.4 Scenario 4: A mistyped target

A user runs `countdown "Aug 19"`. Because the argument does not match `YYYY-MM-DD HH:MM:SS`, the program prints a diagnostic naming the expected form and exits with a non-zero status, without opening a window.

7 Summary of impacts

7.1 Operational impacts

Users gain a reliable, glanceable countdown that runs locally. No new infrastructure, account, or network dependency is introduced.

7.2 Organisational impacts

None of consequence. The program is a single self-contained executable; there is no server to administer and no data to retain between runs.

7.3 Support and maintenance impacts

Because the program is a literate document, maintenance proceeds by reading and editing `countdown.w` and re-tangling. A maintainer needs familiarity with C, Xlib, and the CWEB tools, all of which are documented in the program itself and in CD-SDD-001.

8 Analysis of the proposed system

8.1 Alternatives considered

A terminal (text) countdown. Simplest to build, but the ConOps calls explicitly for a window on the X Window System, and a graphical artefact is more glanceable. Rejected on requirement grounds.

A toolkit-based GUI (GTK, Qt, Motif). Would give buttons and theming for free, but pulls in a large dependency for a one-window program and obscures the underlying mechanism. Rejected in favour of raw Xlib, which keeps the program small, dependency-light, and fully readable.

A browser/web widget. Rejected for the privacy, availability, and dependency reasons set out in Section 4.

8.2 Trade-offs

Choosing raw Xlib trades developer convenience (no ready-made widgets) for a minimal, transparent, portable program. Choosing `select`-based multiplexing over a busy-wait trades a few lines of code for near-zero idle CPU. Choosing local-time semantics (via `mktime`) trades a little ambiguity at daylight-saving boundaries for the convenience of letting users write wall-clock times.

8.3 Justification for the selected approach

The selected approach—a single Xlib window driven by a `select`-based event-and-timer loop, written as a literate program—meets every capability in Section 6 with the least machinery and the greatest readability, and is therefore the most defensible choice for a tool whose whole purpose is to be small, correct, and self-evident.